FINMANAI · PROGRESS REPORT

The Technical Build

Architecture, correctness strategy, security core, and what is genuinely verified as of September 2026.

> Three packages, 22,000 lines of TypeScript, and one organising constraint: the financial engine is a pure function with no I/O, no clock and no dependencies, so it can be tested from a terminal with nothing installed and reproduced byte-for-byte years later. Every correctness claim in this document was re-run against the working tree on the report date, and the commands to repeat each one are given in section 9.

REPORT DATE	ENGINE VERSION	ASSUMPTIONS	RUNTIME
10 September 2026	1.0.0	2026-08	Node 22.6+ · type stripping

STACK

Next.js 15 · React 19 · Prisma 6 · PostgreSQL

1 Architecture

Three packages in one npm workspace, split by how expensive a bug in each would be. The split is enforced by a dependency rule, not by convention.

```
engine/  Pure financial logic. No I/O, no DB, no network, no clock.
         10 stages: normalise → diagnose → risk → waterfall → goals →
         allocation → products → tax → suitability → narrative
         plus rebalance / policy / stress / premiums.

core/    Security primitives. Passwords, TOTP, field encryption,
         audit chain, sessions, profile versioning.

app/     Next.js 15 + Prisma. Thin routes; all logic lives in
         engine/ or app/src/lib/.
```

`engine/` and `core/` have zero runtime dependencies and must stay that way. Both run under Node's built-in TypeScript type stripping, so there is no build step and no `node_modules` required to execute their test suites. That is not minimalism for its own sake: it is what makes the financial logic auditable in five years, when the dependency tree of 2026 may not install. If a package feels necessary in either, it belongs in `app/`.

The constraint that makes the engine testable

`generatePlan()` is a pure function: profile in, plan out, with the date injected rather than read. Four properties follow directly, and all four are load-bearing:

- **Testable without infrastructure.** No database, no fixtures, no server. Every bug listed in section 4 was found by running it from a terminal.
- **Auditable later.** The same profile and the same assumptions version reproduce the same plan indefinitely.
- **Reproducible byte-for-byte.** Which is what makes golden-file testing possible at all.
- **Reusable.** The web app, an adviser console and any batch re-run call the identical function.

Codebase

AREA	LINES	CONTENTS
`engine/src`	5,720	18 modules. Largest: `types.ts` (766), `goals.ts` (525), `rebalance.ts` (524), `narrative.ts` (419), `waterfall.ts` (415)
`engine/tests`	1,144	Invariants, golden files, narrative guards, plus two human-readable reporters
`core/src`	1,081	6 modules: audit chain, field crypto, password, profile store, session, TOTP
`core/tests`	404	103 checks including published RFC test vectors
`app/src`	11,137	26 API routes, 14 pages, 9 components, 19 lib modules
`app/scripts`	2,822	18 operational scripts — smoke, integrity, backup, key rotation, reminders, assumptions staging
`app/prisma`	959	15 models, 4 migrations, seed
Total	**23,267**	**TypeScript, SQL and Prisma schema**

2 The engine

STAGE	MODULE	RESPONSIBILITY
0	`normalise.ts`	Monthly denominators, derived aggregates, per-earner cover need. Variable income is haircut here, before anything downstream can plan on it.
1	`diagnostics.ts`	Ten financial-health ratios with banded thresholds and a written sentence each.
2	`risk.ts`	Capacity scored from seven weighted factors × tolerance from the questionnaire.
3	`waterfall.ts`	Where the next rupee goes, in strict order. **This is the advice.**
4	`goals.ts`	Inflation, required SIP under annual step-ups, feasibility negotiation.
5	`allocation.ts`	Per-goal asset allocation; retirement tranching.
5b	`rebalance.ts`	The existing portfolio: drift measurement, tax-priced switches.
5c	`policy.ts`	ULIP and endowment: surrender vs continue, fully costed.
5d	`stress.ts`	Three deterministic scenarios — job loss, market crash, medical event.
6	`products.ts`	The Indian instrument catalogue, organised asset class → sleeve → instrument category.
7	`tax.ts`	Old vs new regime, deduction headroom, marginal rate.
8	`suitability.ts`	The last gate, and the written SEBI suitability record.
9	`narrative.ts`	LLM prose behind two hard guards. See section 7.
—	`premiums.ts`	Sourced term and health premium tables, stamped `PREMIUM_SOURCE_DATE = '2026-08'`.
—	`assumptions/v2026-08.ts`	Every rate, slab and cap. Versioned and stamped onto each generated plan.

Modelling decisions with technical consequences

Risk profile is `min(capacity, tolerance)`

Capacity is computed from the data across seven weighted factors — years to retirement (25%), income stability (20%), savings rate (15%), emergency fund (15%), dependency burden (10%), leverage (10%), protection (5%). Tolerance comes from the questionnaire. The plan is built to the lower of the two, and where they diverge by more than 20 points the divergence is stated in plain words rather than silently resolved. Most retail products ask how you *feel* about risk and stop there.

Retirement is tranched, not dated

The corpus is split by when each rupee is actually drawn — first three years, years 4–10, year 11 onward — and each tranche is horizoned at years-to-retirement *plus* years-until-spent, planning to age 85. The near tranche carries a separate 10% equity cap for sequence-of-returns risk. For a 29-year-old this produces effective horizons of 30, 35 and 47 years against a nominal 28.6.

Step-up contributions have no closed form

Under annual step-ups there is no closed-form expression for the required starting contribution, so the module computes the accumulation factor for ₹1 numerically and divides. Step-up is `expectedIncomeGrowth × 0.75`, capped at 10%/year — the remaining quarter is assumed absorbed by lifestyle, which is what happens.

Assumptions are versioned data, never inline constants

Nothing in the engine hardcodes a rate. Changing one means copying the dated file, editing the copy, updating the import, running the golden suite, and reviewing every moved figure. **A version is never edited in place**, because plans already generated must stay reproducible against the version stamped on them.

The sensitivity report exists to make the largest assumption arguable: moving expected equity returns from 9% to 13% changes required contributions by roughly 35% on goals beyond fifteen years — but **no goal changes its feasibility verdict anywhere in that range**. The assumption moves the numbers, not the advice. 11.3% blended is held as a defensible middle, with the asymmetry favouring conservatism: over-saving is recoverable, under-saving for thirty years is not.

3 How correctness is defended

The engine produces numbers that are internally consistent and can still be completely wrong. No type system detects that. Three mechanisms do the work instead.

1. Invariants — 33 rules × 9 personas = 297 checks

Structural rules that must hold for every household the engine will ever see. If a change makes one fail, the change is wrong — not the invariant.

#	RULE
1	A household in deficit gets a cashflow-repair plan, never a SIP.
2	High-cost debt above 12% is cleared before goal investing.
3	Protection premiums are a fixed cost, reserved before the waterfall runs — never crowded out by an aggressive debt payoff.
4	Equity never exceeds the risk band's ceiling, *including* after eligibility filtering re-weights the mix.
5	The action plan fits inside the surplus it spends. Every action declares `surplusConsumed`; the total is asserted.
6	Goals inside 24 months hold no equity and no crypto.
7	No instrument locks money past its goal date — retirement excepted, which has its own model.
8	Any underfunded goal carries a written negotiation. Never a silent shortfall.
9	Risk profile is always `min(capacity, tolerance)`.
10	Same input → identical output. The engine is pure; the date is injected.

2. Golden files — 9 full plan snapshots

Complete output snapshots for all nine personas, between 336 and 591 asserted values each. Any change that moves a number produces an explicit diff a human must approve. This is the closest software gets to a regression test for financial advice, and it is what makes changing the engine safe rather than frightening.

The rule attached to it: never accept a golden diff without reading every line. A suite that is approved reflexively is worse than no suite, because it converts a warning into a rubber stamp.

3. Nine personas chosen for the awkward cases

`rohan`, `deficitHousehold`, `firstJob`, `freelancer`, `household`, `nearRetirement`, `overconfident`, `equityAverse`, `overweightEquity` — a household spending more than it earns, a freelancer with lumpy income, a 58-year-old with a ₹1.9 Cr portfolio badly overweight equity, someone who refuses equity entirely, a high earner who is nonetheless overspending.

> THE FAILURE MODE THESE EXIST TO CATCH
>
> Bug 1 in the list below is the instructive one. Dropping an ineligible instrument renormalises the rest; if what was dropped was debt, equity climbs back over the cap. The code *detected* the breach and wrote a record claiming it had moved the excess into debt. It had not. **Prose asserting a fix is not a fix** — which is why the invariant asserts the post-filter weights rather than trusting the remediation log.

4 Nine bugs that shipped, and how they were caught

#	BUG	FOUND BY	STRUCTURAL FIX
1	Equity broke its ceiling after eligibility filtering re-weighted the mix	Invariant test	Assert post-filter weights, not the remediation record
2	Debt payoff treated as permanent — a four-month card payoff zeroed every goal forever	Reading output	Commitments tagged temporary vs permanent; goals planned on steady-state surplus
3	Top-priority goal swallowed the whole surplus, leaving a 29-year-old's retirement at ₹0	Reading output	Funding is proportional *within* a priority tier
4	Action plan committed ₹25,078 of a ₹23,750 surplus, silently clamped to zero, surfaced as “invest ₹0 a month”	**User spotted it**	One field was doing two jobs; split, and the total is now asserted
5	Retirement modelled as a point in time put a 57-year-old's entire corpus in short-duration debt	Writing the review checklist	Spending-stream model with per-tranche horizons
6	`expectedIncomeGrowth` was collected and never read — contributions modelled flat for 30 years, 13 of 14 goals impossible	Grepping for unused inputs	Step-up SIPs
7	13 of 14 goals marked “not achievable” — a modelling artifact, not a finding	Counting labels	Feasibility recomputed against the negotiated plan
8	Crypto fell 3% in the market-crash stress test. It falls harder than equity, not softer	Reading output	Corrected shock factors
9	Module-level mutable state in `rebalance.ts` would leak one household's tax into the next one's plan	Code review	State moved into the call

> THE PATTERN
>
> Internally consistent numbers that are completely wrong. Types caught none of these; four were caught only by a human reading real output for a real persona. That is why `report.ts` exists as a first-class artefact and why reading it is a required step after any engine change, alongside running the suites.

5 The security core

`core/` is where a bug is expensive and silent, so it is built to a different standard: zero dependencies, runnable from a terminal, and validated against published test vectors rather than against itself.

PRIMITIVE	CHOICE	REASONING
Password hashing	**scrypt**, N=2^{15} (32 MiB)	The spec called for argon2id; scrypt is what Node provides without a dependency, and at this cost parameter GPU attacks are not cheap. `PasswordHasher` is an interface, so swapping later touches one file. Parameters live inside the hash string, so raising them does not invalidate existing passwords — `needsRehash()` upgrades users at next login.
Second factor	**TOTP**, validated against RFC 6238 and RFC 4226 vectors	An authenticator implementation nobody checked against the published vectors is a guess that happens to work with one app. `totpLastCounter` is persisted, so a code read over someone's shoulder cannot be replayed inside its 90-second window.
Sessions	**Opaque 256-bit tokens**, SHA-256 stored server-side — not JWT	A deliberate deviation, stated openly. A JWT is valid until it expires because nothing server-side is consulted, so "log out all devices", "this session was hijacked" and "this adviser's access was withdrawn" either fail or need a deny-list — at which point you have a session table anyway, with worse ergonomics. For an application holding a complete financial position under a five-year audit trail, revocability beats statelessness.
Field encryption	**AES-256-GCM** with AAD and key versioning	The profile ID is bound in as additional authenticated data, so an attacker with database write access **cannot move one user's encrypted profile onto another's row** — it will not decrypt. The envelope records which key version encrypted it, so rotation is incremental. Without that, nobody ever rotates.
Login flow	**Two stages; the first issues nothing**	Password success returns a five-minute challenge ticket. The session cookie is set only after TOTP. A stolen password alone never yields an authenticated session. Backoff is exponential.
IP addresses	**Peppered blind index**	Hashed with a pepper, so a stolen database cannot be brute-forced against the small IPv4 space.

The audit chain

```
hash = SHA256(prevHash + action + entityId + createdAt + metadata)
```

This does not prevent tampering. Anyone with database write access can rewrite rows. What it does is make tampering *provable*: rewriting one event breaks every hash after it, and `verifyChain()` reports exactly where. In a dispute, "we cannot prove nobody edited this" and "we can demonstrate nobody edited this" are very different positions.

Three tamper modes are tested, including the subtle one: an attacker who edits an event *and re-hashes it* still breaks the chain, because the next event's `prevHash` no longer matches.

Appends run in a **serialisable transaction**. Two concurrent writers would both compute sequence N+1 and fork the chain, which `verifyChain()` would then report as tampering forever — and a false alarm during an audit is nearly as damaging as a real one. Under contention this is a bottleneck by design.

Every mutation appends an event, including an adviser merely *reading* a client's profile (`adviser.viewed_client`). Subject and actor are recorded separately.

6 Data model and the application layer

Fifteen Prisma models

MODEL	FIELDS	NOTE
`User`	34	Soft delete only — retention forbids hard deletion inside the window
`Session` / `OAuthAccount`	11 / 11	Server-side hashes; Google sign-in state is stored hashed, never raw
`ProfileDraft` / `Profile`	7 / 19	Draft is mutable and autosaved; `Profile` is an immutable version with a SHA-256 of its canonical plaintext
`Plan`	23	Append-only. Stores profile hash, engine version, assumptions version, advisory mode, the encrypted plan, and full narrative provenance
`AuditEvent`	28	Hash-chained, sequenced, subject and actor separate
`Consent` / `ClientAgreement`	17 / 9	Six §15 flags stored individually with consent text version, timestamp, hashed IP, user agent. The agreement stores a hash of the exact MITC document served, not a version string
`ProfileReview`	12	A review is reset by a review, not by a change — confirming “nothing moved” writes no new profile version
`PlanNote` / `PlanApproval`	8 / 8	Notes encrypted; approvals append-only, so a withheld-then-approved sequence stays visible
`AdviserClient`	11	Invite / accept relationship
`Outbox`	14	Mail is queued then sent — a request must not fail because a mail server is slow
`AssumptionsProposal`	13	Stage → review → apply, with no path to it from the web application

Two schema decisions worth defending

The profile is one encrypted JSON column, not fifteen normalised tables. The engine consumes one object; the profile is always read and written whole; the shape is already enforced by TypeScript and Zod. Versioning one column is trivial; versioning fifteen related tables is not. Index only what is queried on.

A plan whose hash no longer matches its profile is not rendered as advice. The page verifies `plan.profileHash === profile.dataHash` before displaying, and `profile-service.ts` throws rather than serving a profile that fails its integrity check. Advice derived from data we cannot vouch for is worse than an error page. `GET /api/plans` returns metadata only and surfaces a `verifiable` flag per plan, so a stale plan is visibly stale — and a list endpoint never decrypts nine plans' worth of financial data to render a sidebar.

The application layer

SURFACE	COUNT	DETAIL
API routes	26	auth (10, incl. Google OAuth and TOTP), profile (4), plan (5), adviser (4), compliance (2), consent, agreement
Pages	14	landing, signup, login, verify-email, TOTP enrol, onboarding wizard + review, dashboard, plan, plan print, plan compare, reprofile, adviser, compliance
Components	9	Including the data-driven `WizardForm`, `SpendingPortrait` and `AllocationChart`
Lib modules	19	Largest: `wizard-config.ts` (627), `validation.ts` (485), `plan-diff.ts` (337), `profile-service.ts` (285)
Operational scripts	18	See section 8
Migrations	4	`init`, `phase6_living_advice`, `google_sign_in`, `audit_events_block_user_deletion`

The eight-step wizard is described as data in `wizard-config.ts` and rendered by one component. Eight hand-written forms would be thousands of lines drifting out of sync with the Zod schema within a month; one renderer means one place to change a label. Validation is shared client and server — never trust the client, never write the rules twice.

All three pre-plan gates live in `plan-service.ts`: consent recorded (both personalised recommendations and risk disclosures), 2FA enabled, and — in advisory mode only — a signed MITC agreement. One place to get right rather than one per route. A future route cannot quietly skip one, because it would have to call the engine directly to do so.

The rate limiter counts the audit log rather than a table of its own: 10 plan generations per hour, counted from events that are already written and already indexed, and that cannot drift out of step with what actually happened. A limiter that counts something other than the thing it limits eventually disagrees with reality.

7 The LLM narrative layer

The model receives the finished plan and writes prose. It cannot add a figure, name a product, re-rank a recommendation or promise anything. Enforcement is not "we asked it nicely" — the system prompt states the rules, but a rule in a prompt is a request. Two independent checks run on every output.

1. Numeric, unit-aware

Every numeral is extracted **with its unit** — ₹ / % / lakh / crore / bare — and checked against the corresponding set of figures the engine actually produced. The unit-awareness was not the first design and is the important part: an earlier version flattened everything into one allowed set, which let a hallucinated **"15.7%"** through because an unrelated rupee amount happened to be ₹15.7 lakh. A rupee figure must not be able to launder itself into a percentage.

The allowed set covers the plan's numeric fields, figures embedded in engine-authored prose (action reasons contain sub-amounts that exist nowhere as a field), the assumptions module, plus sensible roundings and lakh/crore renderings. **A figure altered by ₹1,111 is still caught.**

2. Claim-level

Numbers alone are not enough, and this is the subtler point: a hallucinated "15.7% annual return" can coincide with a genuine 15.7% allocation slice. Identical digits, entirely different meaning; no numeric check can separate them. So phrasing is policed separately. Rejected outright: guarantees and assured returns; certain-future-outcome phrasing ("will reach", "will double"); risk denial ("no risk", "safe bet", "cannot lose"); named fund houses or schemes; and asserted expected returns outside the engine's framing.

One exemption, applied per sentence: a statement that a *debt payoff* is guaranteed is true. Paying off a 42% credit card genuinely is a guaranteed 42% return, and the engine says so. The exemption covers debt, prepayment, PPF, EPF, SCSS and sovereign-backed instruments — and is scoped to the sentence, so an exemption in one cannot excuse a violation in another.

Failure is never fatal

Either check failing discards the narrative and falls back to deterministic template text. So does a missing API key, a timeout or a 502. **The product must never be unable to explain itself because a third-party service is having a bad day.** The fallback is engine-authored prose assembled from the plan's own figures — and it passes its own validator, which is itself a test.

Privacy and provenance

The prompt carries the derived plan, not the profile: no name, no city, no identifiers. The per-earner life-cover breakdown originally leaked real names into the prompt — caught by a test asserting no PII reaches the model, and now redacted to "the primary earner" / "earner 2". The narrative does not need to know who anyone is to explain the numbers.

Every generation records source (`llm` or `fallback`), model, prompt hash, and the rejection reason where one applies. The compliance export aggregates these. The rejection count is the number worth watching: a rising rate means the model is drifting or the prompt needs work. Provider is swappable in one file — `llm-client.ts` is 78 lines of plain `fetch`, and nothing else in the engine knows an LLM exists.

8 Operations

TASK	COMMAND	NOTE
Chain + integrity verification	`npm run checks`	Exits non-zero so cron raises an alarm rather than mailing a log nobody reads. Monthly minimum
Full integrity walk	`npm run verify:integrity`	Re-hashes every profile, confirms every plan is bound to the profile that produced it, flags rows on a retired key — and proves detection works by tampering with an in-memory copy
Review reminders	`npm run send:reminders`	Queue then send, deduped by period, consent re-checked at send time
Key rotation	`npm run rotate:keys`	Re-encrypts in batches; the plaintext hash never changes, so plans stay verifiable across a rotation
Verified backup	`npm run backup -- -- verify`	Restores into a scratch database, counts rows, re-verifies the audit chain *inside the restored copy*, drops the scratch. A dump that has not survived that round trip is reported as unverified
Assumptions staging	`npm run assumptions`	stage → review → apply. `apply` refuses an unreviewed proposal, writes a new dated file, and prints the golden suite so the blast radius is on screen
Read stored data	`npm run inspect -- <email>`	Decrypted profiles and plans, for debugging
Database switching	`npm run db:use <live|test|testdb>`	Three databases, each with its own `FIELD_KEY`, so test ciphertext can never be read as real ciphertext

> TWO GUARDS WORTH THEIR OWN SENTENCE
>
> **The backup verifies by restoring.** An untested restore is not a backup. The failure people actually have is not a missing dump — it is a dump that has been running nightly for two years and cannot be loaded, discovered on the worst possible day.
>
> **`npm run smoke` and `npm run db:seed` hard-refuse any database whose name lacks "test".** They write append-only plans and hash-chained audit events, and there is no clean way to remove those afterwards.

Two failure modes that look like broken code and are not

- **`Cannot read properties of undefined (reading 'findFirst')`** — the dev server is holding a Prisma client generated before the last migration. Restart it.
- **`ENOENT … .next/server/app/…/route.js`** — more than one `next dev` is running against the same `.next` directory, clobbering each other's compiled output. `pkill -f "next dev" && rm -rf app/.next`, then start exactly one.

Operational procedure lives in `RUNBOOK-OPERATIONS.md`; incident procedure in `RUNBOOK-BREACH.md`, which was written before it was needed. Its most important instruction: **if the chain breaks, preserve it, never repair it.** A chain that has been "fixed" proves nothing, and is afterwards indistinguishable from an attacker having done the same.

9 Verification — results and how to repeat them

Everything below was run against the working tree on 10 September 2026. The commands are given so each result can be reproduced independently.

Run in this session — all passing

COMMAND	RESULT	WHAT A PASS MEANS
`node --experimental-strip-types core/tests/core.ts`	**103 / 103**	Password hashing and rehash detection, TOTP against RFC vectors and replay rejection, recovery codes, encryption round-trip and AAD binding, tamper detection, key rotation, session lifecycle, profile versioning
`node --experimental-strip-types engine/tests/invariants.ts`	**297 / 297**	33 invariants across all 9 personas. Prints "297 passed, 0 failed across 9 personas × 33 invariants"
`node --experimental-strip-types engine/tests/narrative.ts`	**16 / 16**	Numeric and claim guards, PII absence from the prompt, clean LLM-outage fallback
`node --experimental-strip-types engine/tests/golden.ts`	**9 / 9**	"9 persona(s) unchanged, 0 written." **Any other number means a plan moved** — read every line of the diff before accepting it
`cd app && npm run test:phase6`	**50 / 50**	Review clock at its boundaries, material-change detection, plan-diff noise suppression, deposit bands, OAuth state handling. No database required
`cd app && npx tsc --noEmit`	**clean**	Exit 0, no output. The whole application layer compiles
Offline total	**475**	**Runs in seconds, needs nothing installed for the engine and core suites**

Not re-run in this session — requires a running server and database

COMMAND	DOCUMENTED	PREREQUISITES
`cd app && npm run smoke:full`	72 checks	Needs `npm run dev` running against a *test* database. Drives signup → 2FA → eight wizard steps → commit → plan → read-back over HTTP, and asserts the advisory gate is consistent
`cd app && npm run verify:integrity`	—	Needs a populated database. Reports chain integrity, profile hashes, plan bindings and key versions

TO REPRODUCE EVERYTHING, IN ORDER

```
# 1. The four offline suites — nothing to install, seconds to run
node --experimental-strip-types core/tests/core.ts
node --experimental-strip-types engine/tests/invariants.ts
node --experimental-strip-types engine/tests/narrative.ts
node --experimental-strip-types engine/tests/golden.ts

# Expect, respectively:
#   103 passed, 0 failed
#   297 passed, 0 failed across 9 personas × 33 invariants
#   16 passed, 0 failed
#   9 persona(s) unchanged, 0 written.

# 2. Read the actual advice — four of the nine bugs were found only this way
node --experimental-strip-types engine/tests/report.ts rohan
node --experimental-strip-types engine/tests/sensitivity.ts

# 3. The app layer
cd app
npx tsc --noEmit          # expect: no output, exit 0
npm run test:phase6       # expect: 50 passed, 0 failed

# 4. End to end over HTTP — test database only
npm run db:use test
npm run dev               # in one terminal; exactly one
npm run smoke:full        # in another; expect 72 checks passing

# 5. Data integrity, against a populated database
npm run verify:integrity  # expect: chain intact, 0 unverifiable plans
```

Use `smoke:full`, not bare `smoke`. The bare one mints a fresh account each run, and a fresh account is unverified, so it always stops at the email-verification gate — that is the product working, not a failure. `smoke:full` pins one address and does the verify-and-resume itself.

10 Technical debt and open items

ITEM	STATUS	DETAIL
Live data adapters	STAGED	The stage → review → apply pipeline is built and the web application deliberately has no path to it. The network fetchers (AMFI NAV, gold spot, MoSPI CPI) are not built; they need credentials and live endpoints. Sources and refresh cadences are documented in the operations runbook
Narrative against a live model	UNTESTED	Every test uses a stub. The guards are proven; the prose quality is not. Temperature is 0.2 and untuned
Premium table staleness	AGEING	Stamped `2026-08`. Premiums are subtracted from surplus *before* anything is invested, so an error distorts the whole plan. An invariant asserts they stay in a plausible range and rise with age — that catches gross drift, not gentle staleness. Due for annual re-check
Assumptions staleness	AGEING	Tax slabs change at each Budget; small-savings rates quarterly. Version `2026-08` is one year old at the report date
Audit-chain write throughput	BY DESIGN	Serialisable transactions serialise appends. Correct, and a bottleneck under contention. Not a problem at pilot scale; it will need attention before it is one
Monte Carlo for retirement	DEFERRED	Three deterministic stress tests. A sequence-of-returns simulation would strengthen the model
Documentation drift	MINOR	The phase documents and `CLAUDE.md` record 36 living-advice checks; the suite now runs 50, having grown with the Google sign-in work that postdates the write-ups. `CLAUDE.md` also lists plan charts as not built, but `AllocationChart` is implemented and rendered on the plan page. Both should be reconciled
Version control	ATTENTION	The working tree is a git repository on `main` with **no commits**. All 23,000 lines are uncommitted and unbacked by history — there is no diff to review, no bisect available, and no recovery from an accidental overwrite. This is the cheapest item on the list to fix and the most expensive to leave

The three files to read first

For anyone picking this up with limited time, in this order:

1. `engine/src/waterfall.ts` — the sequencing decisions. This *is* the advice.
2. `engine/src/goals.ts` — the maths, and the negotiation when a goal does not fit.
3. `app/src/lib/profile-service.ts` — it throws rather than serving a profile that fails its integrity check.

> THE ONE THING NOT TO WEAKEN
>
> Every phase after the engine depends on the engine being provably right, because none of them can check it. The golden-file suite is what makes changing it safe: any change that moves a number for any of the nine households produces an explicit diff a human must approve. If one thing in this repository is ever weakened, do not let it be that.